Pairing Proofs and Polynomial Ring Toolkit

Revision requirements and responses — Major Revision resubmission

Resubmission: TCHES 2027, Issue 1, Paper #185 · **Original submission:** TCHES 2026, Issue 4, Paper #104 (Major Revision)

All modified or added parts are **highlighted in yellow** in the revised paper. The original submission PDF accompanies this document. Section/table/algorithm numbers refer to the revised paper; note the prover and verifier algorithms are now Algorithms 3 and 4 (two auxiliary algorithm floats were removed).

Part A addresses the binding revision criteria (Comment @A1); **Part B** the remaining individual review points. Only changes made in this revision are listed.

Part A — Binding revision criteria (Comment @A1)

> **A.1** *Clarify the main contribution relative to the most closely related prior works (such as Fel23 and Garaga)*

- Section 3 now quantifies the delta over [Fel23]: a 3-pair zero-bit Miller loop step takes four chained relations and 48 F_q commitments under [Fel23] versus one relation and 12 commitments in our scheme (derivation in Section 4.1).
- New closing paragraph of Section 3 on **Garaga**: it deployed an early Cairo-specific proof-of-concept in 2024, adapted from a preceding technical write-up (cited anonymously), with no formal treatment; this paper supplies the foundations (Sections 4–5) and the systematic evaluation against gnark's state-of-the-art baseline. Anonymity is unaffected.

> **A.2** *Add additional benchmarks for BLS12-381, as per the rebuttal*

- The toolkit is now implemented for BLS12-381 (`sw_bls12381`) and Table 7 adds full BLS12-381 results for both SCS and R1CS: 33.2% / 32.5% fewer constraints, 54.5% fewer committed elements, up to 48.8% less solver memory. Abstract, Sections 1.2 and 2.1 updated accordingly.

> **A.3** *Clarify the benchmarking setup, in particular the 3-pairing setting*

- We replaced the original composite "Groth16 simulation" circuit (2-pair fixed-G2 check plus accumulated single pairing) with one uniform benchmark: a 3-pairing product check $\prod_{i=1..3} e(P_i, Q_i) = 1$ via `PairingCheck`, identical for baseline and our fork. The rewritten "Benchmark circuit" paragraph (Section 7.2) explains why this is the Groth16 verification workload (the constant $e(\alpha,\beta)$ folds into a product over three variable pairs) and notes the same 3-pairing configuration is used in the CairoVM benchmarks (Table 6). All numbers regenerated (abstract, Section 1.2, Table 7).
- New text in Section 7.2 states the baseline's provenance (upstream gnark `emulated` package with the [Hou23] and [NE24] optimizations) and that both implementations exploit Miller line sparseness; Section 4.1 now states the sparseness savings are included in all constraint counts.

> **A.4** *Improve the mapping between algorithms and implementation, in particular the application of Fiat-Shamir*

- Section 5.5.1 (Non-Interactive Proof) rewritten: the transcript is now explicit — z = H(pairing inputs, c, W^w, $\mathscr{R}$), x = H(z, Q_{acc}) — with a new argument for why Theorem 1's soundness carries over (each prover-chosen value is absorbed before the challenge it must not depend on). The redundant non-interactive protocol figure was removed.
- Section 6.3 ("Algorithms and Implementation", expanded from the former API subsection) now traces Algorithms 3/4 onto the code: `MulPolyRings`/`polyRingMulHint` execute the prover's work during witness generation; `performDeferredRingChecks` implements Equation (5) via two sequential `api.Commit` calls (producing z, then x) plus one `AssertIsEqual` per ring group — the verifier's work. New subsections "Shared Circuit Skeleton" and "Prover's and Verifier's Work"; new `ToCommit` API entry for hinted values absorbed before z.

> **A.5** *Qualify the commitment-based performance metric: schemes, overhead, impact*

- New Section 5.5.2 (Instantiating the Random Oracle): CairoVM commits via a Poseidon sponge over calldata hints — hashing cost and calldata scale linearly with committed elements (why COMMITS accompanies modular ops in Table 6); gnark commits via `api.Commit` ([BSB23]/[BSB]) with backend-specific costs spelled out — Groth16: two G_1 MSMs and one extra G_1 proof element per commitment, no constraint-count impact; PLONK: one constraint per committed wire, evaluation folded into the existing batched KZG opening.
- Metric definitions tightened: Table 6's COMMITS footnote now cross-references Section 5.5.2; a new Section 7.2 paragraph defines Table 6's Before/After columns and all four metrics; Table 7's metric is renamed "Nb F_q Committed" (elements committed via `api.Commit`); commitment terminology unified throughout.

> **A.6** *Minor presentation issues pointed out by the reviewers*

All addressed — see Part B.

Part B — Individual review points

Review 104A

> *Why BN254 and not a modern alternative? Add BLS12-381.*

BLS12-381 added throughout (A.2). BN254 is kept alongside as it remains ubiquitous in deployed systems (Ethereum precompiles and rollups built on them).

> *Relationship with Garaga; novelty over it.*

A.1: new Section 3 paragraph and clarified Section 7.2.

> *Do you exploit Miller line sparseness?*

Yes — both compared implementations do; now stated explicitly in Sections 4.1 and 7.2.

> *What motivates the 3-pairing benchmark? Same as the Section 7.2 Groth16 circuit? Metrics for single/n pairings?*

Yes, it is the Groth16 pairing workload; the benchmark is now one uniform 3-pairing product check for both implementations (A.3). For other pair counts, per-step costs (Section 4) scale via deg(P) ≈ 88 + 18(k−3) (footnote, Section 4.3).

> *Redundant floats; combine Algorithms; interactive vs non-interactive versions similar.*

Removed the PolyMulDiv and EvalPoly algorithm floats (their signatures are inlined into Algorithms 3/4) and the separate non-interactive protocol figure (now three lines of text in Section 5.5.1). The prover/verifier algorithms stay separate floats because Section 6.3 uses them to anchor the implementation mapping, with their shared structure made explicit there.

Review 104B

> *1. Contribution vs prior work. 2. Interactive proof / Fiat-Shamir presentation and gnark relation. 4. Metrics and benchmarking setup; commitment metrics.*

See A.1; A.4; A.3 + A.5 respectively.

> *3. Clean up notation and terminology.*

Commitment terminology unified ("committed elements", Table 7 metric renamed); polynomials now use the formal indeterminate X consistently, keeping z, x $\in F_q$ as scalars (Section 5.6).

> *How does the protocol map to the gnark implementation in the Fiat-Shamir setting?*

A.4 — Section 6.3 traces it call-by-call; the two `api.Commit` calls realise the two challenges of Section 5.5.

Review 104C

> *Table 6: what do "Before"/"After" compare relative to this work, Fel23, Hou23, NE24?*

Now defined in Section 7.2: *Before* = Garaga verifying each extension-field multiplication individually via [Fel23]; *After* = Garaga with our consolidated relations. Deltas vs [Hou23]/[NE24] are in Section 3.

> *Be precise about the Table 7 gnark baseline's state vs recent academic results.*

Stated in Section 7.2: upstream gnark `emulated` package including the [Hou23] and [NE24] optimizations; identical benchmark and harness for both runs.

> *MULMOD, ADDMOD, COMMITS, VM STEPS never properly defined.*

Defined in the new Section 7.2 metrics paragraph, with the COMMITS footnote cross-referencing Section 5.5.2.

> *Toolkit not evaluated beyond the pairing use case.*

The toolkit is now framed as a reusable artefact evaluated via the pairing workload; Section 6 documents its generic interface (arbitrary modulus, multiple ring groups) and a new note in Section 6.5 covers dynamic overflow tracking beyond the BN254 instantiation. A multi-workload evaluation is left as future work.

> *29%/60%/42% repeated; broken sentence at 178–179; x as both scalar and formal indeterminate (258–259); API description reads like library documentation.*

Respectively: headline numbers now appear only in the abstract and Section 1.2 (updated to the new benchmark); the broken sentence is fixed (Section 4.2); the scalar/indeterminate convention is fixed and applied (Section 5.6); the API text was condensed and repurposed into the algorithm-to-implementation mapping of Section 6.3 (kept in the main body since binding criterion A.4 is discharged there).

Summary of changes (all highlighted in yellow in the revised paper)

Location	Change	Criteria
Abstract, §1.2, §2.1	Updated headline numbers: uniform 3-pairing benchmark, BLS12-381 added	A.2, A.3
§3	Concrete Fel23 comparison; new Garaga paragraph	A.1
§4.1, §4.2	Sparseness statement; fixed benchmark cross-reference sentence	A.3, A.6
Algorithms 3, 4	Inlined PolyMulDiv/EvalPoly signatures (two floats removed)	A.6
§5.5.1	Explicit Fiat-Shamir transcript and soundness carry-over (replaces NI figure)	A.4
§5.5.2 (new)	Random-oracle instantiations with per-scheme commitment costs	A.5
§5.6	Scalar vs. indeterminate notation applied	A.6
§6.3	Expanded API + new algorithm-to-implementation mapping subsections; ToCommit	A.4
§6.5	Dynamic overflow-tracking note	—
§7.2	Before/After & baseline definitions, metrics paragraph, rewritten benchmark circuit; Table 7 regenerated (BLS12-381, uniform circuit)	A.2, A.3, A.5